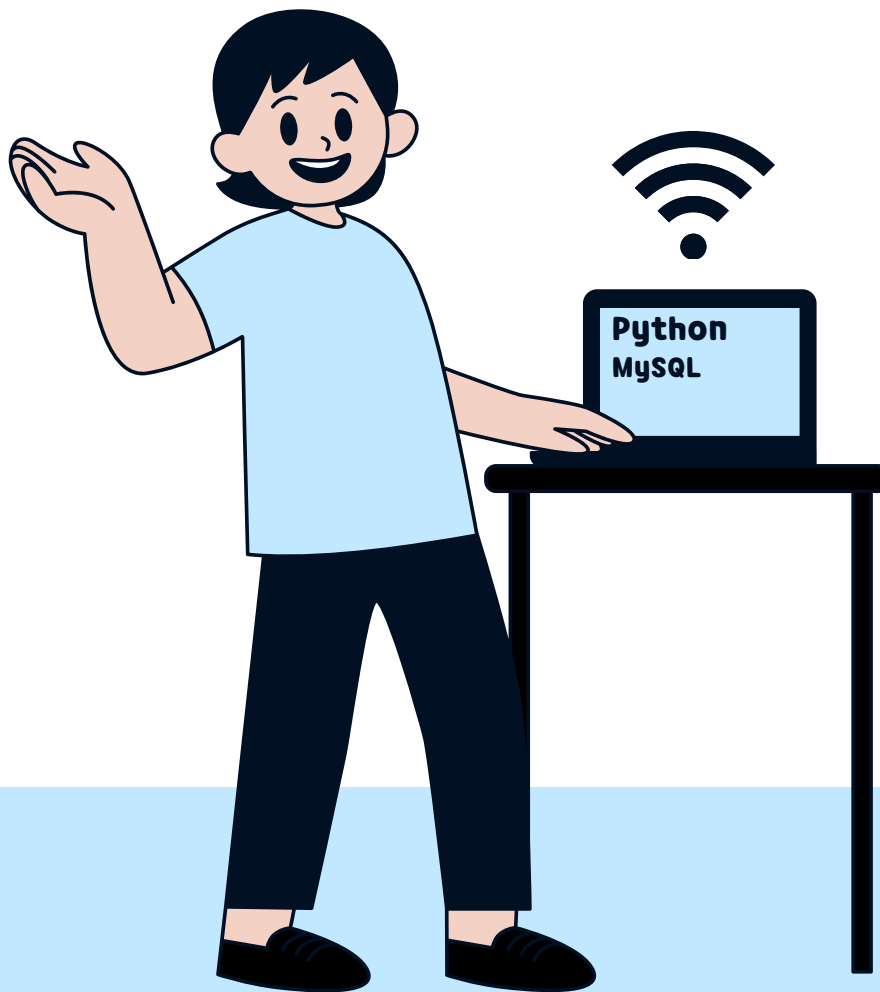




**SHRI SHANKARACHARYA
VIDYALAYA HUDCO BHILAI**

Computer Science Project (Railway Reservation System)



Name - Vikrant Kumar (12th/D)
Roll No. - 30612668

Guided By - Krishna Kumar Chandra Sir

Index

Topics	Page No.
Overview Of Python and MYSQL	1 – 4
Summary of our project	5 – 7
Need for the Project	7 – 8
User Manual	8 – 10
Modules and Functions	11 – 14
Databases and Tables	14
Codes	15 – 20
Output	21 – 24
Limitations	25
Reference	26

1) Overview Of Python and MySQL

1. What is Python?

Python is a high-level, interpreted, object-oriented, and general-purpose programming language. It is known for simple syntax and readability.

2. Key Features Of Python :-

- **Easy to learn**
- **Interpreted (line-by-line execution)**
- **Dynamically typed**
- **Supports object-oriented programming**
- **Huge standard library**
- **Portable and platform-independent**

(A) Functions Of Python :-

- **User-defined functions: created using def**
- **Arguments: positional, default, keyword, variable-length.**
- **Return statement**
- **Scope: local and global variables (global keyword)**

(B) File Handling Of Python :-

Used to store data permanently.

File types:

- **Text files**
- **Binary files**

Modes:

- **'r', 'w', 'a', 'rb', 'wb', 'ab'**

Key functions:

open(), read(), write(), readline(), close()

3. What is MySQL?

MySQL is a Relational Database Management System (RDBMS) used for storing and managing structured data.

4. Key Terms Of MySQL :-

- **Database: Collection of related tables**
- **Table: Rows + Columns**
- **Row (Record)**
- **Column (Field)**
- **Primary Key: uniquely identifies each record**
- **Foreign Key: links one table to another**

(A) Data Types

- **INT, FLOAT, DECIMAL**
- **CHAR, VARCHAR**
- **DATE, TIME, DATETIME**

(B) Basic SQL Commands

- **DDL (Data Definition Language)**
CREATE TABLE, DROP TABLE and ALTER TABLE
- **DML (Data Manipulation Language)**
INSERT, UPDATE and DELETE
- **DQL (Data Query Language)**
SELECT with: WHERE, LIMIT, ORDER BY, GROUP BY and HAVING.

(C) Constraints

- **PRIMARY KEY**
- **UNIQUE**
- **NOT NULL**
- **FOREIGN KEY**

(D) Joins

Used to fetch data from multiple tables.

- **NATURAL JOIN**
- **EQUI JOIN**

(E) Functions

1. String Functions

LENGTH(), LOWER(), UPPER(), SUBSTRING()

2. Math Functions

ABS(), MOD(), ROUND()

3. Date Functions

CURDATE(), NOW(), YEAR(), MONTH()

4. Aggregate Functions

COUNT(), MAX(), MIN(), AVG(), SUM()

5.) Python – MySQL Connectivity

Steps:

1. Import connector → import mysql.connector

2. Establish connection

3. Create cursor

4. Execute query

5. Fetch results

6. Close connection

2) Summary of our project

1. DATABASE SETUP :-

- **Connects to MySQL using mysql.connector.**
- **Creates database railway (if not present).**
- **Creates three tables:**
 - **users – stores registration details**
 - **trains – stores train information**
 - **bookings – stores ticket booking records**

2. PRE-LOADED TRAIN DATA :-

Three trains are inserted:

- **Rajdhani Express**
- **Shatabdi Express**
- **Duronto Express**

Each with source, destination, and seat availability.

3. FUNCTIONAL MODULES :-

A. User Management

1. Sign Up :-

- **Takes username, first/last name, password, email, phone number.**
- **Validates phone number format.**
- **Captcha verification for security.**
- **Inserts user into database.**

2. Sign In

- Checks credentials against DB.
- Captcha verification.
- If successful, user enters Railway Services menu.

B. Train Operations :-

View All Trains – Displays all trains with route and available seats.

Search Train by Route – Finds trains for a given source -> destination.

C. Ticket Booking Workflow :-

- User enters Train No and seats.
- Checks seat availability.
- Fare calculation.
- Payment gateway simulation:
 - Confirm -> payment successful
 - Else -> cancel
- On success:
 - PNR generated (random 10 chars)
 - Booking inserted into database
 - Seats updated in trains table

D. Booking Management :-

- View My Bookings

1. **Shows:** Booking ID, PNR, Train, Source -> Destination, Seats booked and Booking timestamp.

- 2. PNR Status :** Displays booking details for a given PNR.
- 3. Cancel Ticket :** Deletes booking, Updates seats back to train and Refund calculated.

E. UI Structure :-

- **ASCII-based menu system**
- **Easy navigation**
- **Loop-based retry and error handling**

3) Need for the Project

In today's world, computerized reservation systems are essential for managing large-scale passenger bookings efficiently. Manual railway reservation leads to problems such as long queues, delays, human error, and difficulty in retrieving passenger information.

To overcome these limitations, an automated system is required.

This project “Railway Reservation System using Python and MySQL” is developed to provide:

- **Easy Registration & Login for users**
- **Quick Search of Trains based on source and destination**
- **Instant Ticket Booking with automated PNR generation**

- **Real-time Seat Availability Management**
- **User-friendly Interface for viewing and cancelling bookings**
- **Secure Database Storage using MySQL**

This project demonstrates how Python and MySQL can be integrated to build real-life applications. It also helps in understanding database connectivity, SQL operations, data validation, and backend logic.

Thus, the project fulfills the need of creating a small yet functional railway reservation module that showcases database-driven application development in Python.

3) User Manual

1. HARDWARE REQUIREMENTS :-

Components	Mininium Requirements
Processor	Intel/AMD Dual Core or above
RAM	2 GB (4 GB recommended)
Storage	200 MB free disk space
Keyboard	Standard keyboard for input
Display	1024x768 resolution or higher

1. SYSTEM REQUIREMENTS :-

Software	Version / Description
Operating System	Windows 10/ 11 / Linux / macOS
Python	Python 3.8 or above
MySQL Server	MySQL 5.7 / 8.0
Python Libraries	mysql-connector-python and random
Code Editor	Python - IDLE

3. INSTALLATION GUIDE :-

Step 1: Install Python

1. Go to: <https://www.python.org/downloads>
2. Download latest Python 3.x installer.
3. While installing, tick "Add Python to PATH".
4. Finish installation.

Check installation using: (python --version) command on windows command prompt.

Step 2: Install MySQL Server

1. Download MySQL Installer:

<https://dev.mysql.com/downloads/installer/>

2. Install the following components:

- MySQL Server
- MySQL Workbench (optional)

- **During installation:**

- **Set username = root**
- **Set password -> (Remember this! You will enter this when running the project)**

To check if MySQL is running: (mysql -u root -p) command in windows command prompt.

Step 3: Install Required Python Library

Open CMD and run: pip install mysql-connector-python

Step 4: Create a Folder and Save our Project File

Example:

Railway_Project/railway_reservation.py

Paste our complete code in this file using Python IDLE

3. HOW TO RUN THE PROJECT :-

- **Open Python IDLE**
- **Navigate our project folder and open .py file**
- **Run the File**

4) Modules and Functions :-

MODULES :-

1. mysql.connector (imported as sqltor) :-

- Used for connecting Python with MySQL database
- Performs CRUD operations (INSERT, SELECT, UPDATE, DELETE)

2. random :-

- Used for captcha generation
- Used for generating random PNR numbers

FUNCTIONS :-

INTERNAL FUNCTIONS (Utility Functions)

1. validate_phone_no(phone_number)

- Removes spaces/hyphens
- Checks if phone number has exactly 10 digits
- Returns True/False

2. check_no(no)

- Calls validate_phone_no()
- Sets a global variable Phone_Number_Error for validation

3. generate_captcha(length=6)

- Creates a 6-character random alphanumeric captcha
- Used in login and signup to prevent bots

4. verify_captcha()

- Shows a captcha
- User must type it exactly
- Returns True/False

5. generate_pnr()

- **Creates a 10-character random PNR code**
- **Used in ticket booking**

6. payment_gateway(amount)

- **Simulates a payment process**
- **Asks user to confirm**
- **Returns True if user says "yes"**

TRAINS FUNCTIONS

1. SignUp()

- **Takes username, name, password, email, phone**
- **Validates phone number**
- **Verifies captcha**
- **Inserts new user into database**
- **Handles duplicate username errors**

2. SignIn()

- **Accepts username + password**
- **Verifies captcha**
- **Checks credentials in database**
- **On success -> enters railway_services() menu**

3. view_trains()

- **Displays all trains with:**
 - **Train No**
 - **Train Name**
 - **Route**
 - **Seats Available**

4. search_trains()

- Takes source & destination
- Shows all trains matching route

5. book_ticket(user_name)

- Takes train number & seats to book
- Checks seat availability
- Calculates fare ($\text{₹}500 \times \text{seats}$)
- Runs payment gateway simulation
- Generates PNR
- Inserts booking
- Updates seat count in trains table

6. view_bookings(user_name)

- Shows all bookings for that user
- Includes: Booking ID, PNR, Train Name, Route, Seats and Time

7. pnr_status()

- Takes a PNR
- Displays complete booking details
- Shows a clean status output

8. cancel_ticket(user_name)

- Takes PNR
- Checks if booking belongs to logged-in user
- Removes booking from database
- Adds seats back to train
- Shows refund amount

MAIN MENU FUNCTION

9. railway_services(user_name)

- **Displays services menu: View Trains, Search Train, Book Ticket, View Bookings, PNR Status, Cancel Ticket and Logout.**
- **Based on user input, calls relevant functions**

5) Databases and Tables :-

DATABASE: RAILWAY

(Automatically created if not available)

Tables:

- 1. users -> Stores signup details**
- 2. trains -> Preloaded train information**
- 3. bookings -> Stores ticket booking records with timestamps**

6) Full Code :-

```
##### DB CONNECTIONS #####

passwd = str(input("Enter Database Password: "))
myconn = sqltor.MySQLConnection(host="localhost", user="root", password=passwd)
myconn.autocommit = True
cursor = myconn.cursor()

##### DB & TABLE CREATIONS #####

cursor.execute("CREATE DATABASE IF NOT EXISTS railway")
cursor.execute("USE railway")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        User_Name VARCHAR(100) PRIMARY KEY,
        First_Name VARCHAR(100),
        Last_Name VARCHAR(100),
        Password VARCHAR(50),
        Email_Id VARCHAR(50),
        Phone_No VARCHAR(20)
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS trains (
        Train_No INT PRIMARY KEY,
        Train_Name VARCHAR(100),
        Source VARCHAR(100),
        Destination VARCHAR(100),
        Seats_Available INT
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS bookings (
        Booking_ID INT AUTO_INCREMENT PRIMARY KEY,
        User_Name VARCHAR(100),
        Train_No INT,
        Seats_Booked INT,
        PNR VARCHAR(10) UNIQUE,
        Booking_Time DATETIME DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (User_Name) REFERENCES users(User_Name),
        FOREIGN KEY (Train_No) REFERENCES trains(Train_No)
    )
""")

##### TRAINS DETAILS #####

trains = [
    (1001, "Rajdhani Express", "Delhi", "Mumbai", 100),
    (1002, "Shatabdi Express", "Delhi", "Chandigarh", 75),
    (1003, "Duronto Express", "Mumbai", "Kolkata", 50)
]
for train in trains:
    cursor.execute("INSERT IGNORE INTO trains VALUES (%s, %s, %s, %s, %s)", train)

print("✅ Database setup complete.")

##### INTERNAL FUNCTIONS #####

def validate_phone_no(phone_number):
    cleaned_no = phone_number.strip().replace("-", "").replace(" ", "")
    check_ten_digits = len(cleaned_no) == 10
    digits = cleaned_no.isdigit()
    return check_ten_digits and digits
```

```

def check_no(no):
    valid = validate_phone_no(no)
    if valid:
        print()
    else:
        global Phone_Number_Error
        Phone_Number_Error = 1
        print()

def generate_captcha(length=6):
    chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
    return ''.join(random.choice(chars) for _ in range(length))

def verify_captcha():
    captcha = generate_captcha()
    print("\nCaptcha:", captcha)
    user_input = input("Enter the captcha as shown above: ").strip()
    if user_input == captcha:
        print("✅ Captcha verified!")
        return True
    else:
        print("❌ Incorrect captcha.")
        return False

def generate_pnr():
    return ''.join(random.choices('ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789', k=10))

def payment_gateway(amount):
    print(f"\n💰 Payment Gateway: Amount to pay ₹{amount}")
    confirm = input("Proceed with payment? (yes/no): ").lower()
    if confirm == "yes":
        print("Processing payment...")
        print("✅ Payment successful!")
        return True
    else:
        print("❌ Payment cancelled.")
        return False

```

```

##### MAIN FUNCTIONS #####
def SignUp():
    while True:
        global Phone_Number_Error
        Phone_Number_Error = 0
        width = 45
        line = "=" * width

        print("+ " + line + "+")
        print("| " + " SIGN UP PAGE ".center(width) + "|")
        print("+ " + line + "+")

        print()
        print("Type 'quit' as the username to leave the sign-up page.")
        print()

        user_name = input("Username: ")

        if user_name.lower() == "quit":
            print("Exiting Into Sign In Page!")
            break

        first_name = input("First Name: ")
        last_name = input("Last Name: ")
        password = input("Password: ")
        email_id = input("Email: ")
        phone_no = (input("Phone Number: "))

        check_no(phone_no)

        if Phone_Number_Error == 1:
            print("❌ Invalid Phone Number, Try Again!")
            print()
            continue
        else:
            if not verify_captcha():
                print("❌ Incorrect captcha.")
                print()
                continue

            command = "INSERT INTO users VALUES (%s, %s, %s, %s, %s, %s)"
            values = (user_name, first_name, last_name, password, email_id, phone_no)
            try:
                cursor.execute(command, values)
                print()
                print("✅ Account created successfully!")
                break
            except sqlite3.IntegrityError:
                print("❌ Data Entry Error!")
                print()
                continue

```

```

def SignIn():
    while True:
        width = 45
        line = "=" * width

        print("+ " + line + "+")
        print("|" + " SIGN IN PAGE ".center(width) + "|")
        print("+ " + line + "+")

        print()
        print("Type 'quit' as the username to leave the sign-in page.")
        print()

        user_name = input("Username: ")

        if user_name.lower() == "quit":
            print("Exit Into Sign In Page!")
            break

        password = input("Password: ")

        if not verify_captcha():
            print("✗ Incorrect captcha.")
            print()
            continue

        cursor.execute("SELECT * FROM users WHERE User_Name = %s AND Password = %s", (user_name, password))
        record = cursor.fetchone()

        if record:
            print()
            width = 47
            line = "=" * width

            print("+ " + line + "+")
            print("|" + f" [✓] Welcome back, {record[1]} {record[2]}! ".center(width - 1) + "|")
            print("+ " + line + "+")
            print()
            railway_services(user_name)
            break
        else:
            print("✗ Invalid username or password.")
            continue

def view_trains():
    cursor.execute("SELECT * FROM trains")
    rows = cursor.fetchall()
    print(f"{'Train No':<10} {'Train Name':<20} {'Route':<25} {'Seats':<5}")
    print("-" * 65)

    for train in rows:
        route = f"{train[2]} → {train[3]}"
        print(f"{train[0]:<10} {train[1]:<20} {route:<25} {train[4]:<5}")

    print()

def search_trains():
    source = input("Enter Source: ")
    destination = input("Enter Destination: ")
    cursor.execute("SELECT * FROM trains WHERE Source = %s AND Destination = %s", (source, destination))
    results = cursor.fetchall()
    if results:
        print("\nMatching Trains:")
        for train in results:
            print(f"{train[0]} | {train[1]} | Seats: {train[4]}")
        print()
    else:
        print("✗ No trains found.")

```

```

def book_ticket(user_name):
    train_no = int(input("Enter Train No: "))
    seats = int(input("Seats to book: "))
    cursor.execute("SELECT Train_Name, Seats_Available FROM trains WHERE Train_No = %s", (train_no,))
    train = cursor.fetchone()
    if not train or train[1] < seats:
        print("✗ Train not found or insufficient seats.")
        return
    fare = seats * 500
    if payment_gateway(fare):
        pnr = generate_pnr()
        cursor.execute("""
            INSERT INTO bookings (User_Name, Train_No, Seats_Booked, PNR)
            VALUES (%s, %s, %s, %s)
            """, (user_name, train_no, seats, pnr))
        cursor.execute("UPDATE trains SET Seats_Available = Seats_Available - %s WHERE Train_No = %s", (seats, train_no))
        myconn.commit()
        print()
        print(f"📄 Ticket booked. Your PNR: {pnr}")
        print()
    else:
        print("✗ Payment failed. Booking cancelled.")
        print()

def view_bookings(user_name):
    cursor.execute("""
        SELECT b.Booking_ID, b.PNR, t.Train_Name, t.Source, t.Destination, b.Seats_Booked, b.Booking_Time
        FROM bookings b JOIN trains t ON b.Train_No = t.Train_No
        WHERE b.User_Name = %s
        """, (user_name,))
    rows = cursor.fetchall()
    print()
    if rows:
        print(f"{'Booking ID':<12} {'PNR':<12} {'Train':<20} {'From':<12} {'To':<15} {'Seats':<6} {'Time'}")
        print("-" * 90)
        for b in rows:
            print(f"{b[0]:<12} {b[1]:<12} {b[2]:<20} {b[3]:<12} {b[4]:<15} {b[5]:<6} {b[6]}")
        print()
    else:
        print("No bookings found.")
        print()

def pnr_status():
    pnr = input("Enter your PNR: ").strip().upper()
    cursor.execute("""
        SELECT b.PNR, t.Train_Name, t.Source, t.Destination, b.Seats_Booked, b.Booking_Time
        FROM bookings b
        JOIN trains t ON b.Train_No = t.Train_No
        WHERE b.PNR = %s
        """, (pnr,))
    result = cursor.fetchone()
    if result:
        print("\n📄 PNR Status:")
        print(f"PNR: {result[0]}, Train: {result[1]}, From: {result[2]}, To: {result[3]}, Seats: {result[4]}, Time: {result[5]}")
        print()
    else:
        print("✗ Invalid PNR.")
        print()

def cancel_ticket(user_name):
    pnr = input("Enter PNR to cancel: ").strip().upper()
    cursor.execute("SELECT Train_No, Seats_Booked FROM bookings WHERE PNR = %s AND User_Name = %s", (pnr, user_name))
    result = cursor.fetchone()
    if not result:
        print("✗ Booking not found or not yours.")
        return
    train_no, seats = result
    refund = seats * 500
    confirm = input(f"Confirm cancel? Refund ₹{refund} (yes/no): ").lower()
    if confirm == "yes":
        cursor.execute("DELETE FROM bookings WHERE PNR = %s", (pnr,))
        cursor.execute("UPDATE trains SET Seats_Available = Seats_Available + %s WHERE Train_No = %s", (seats, train_no))
        myconn.commit()
        print()
        print(f"✅ Cancelled Sucessfully. Refund ₹{refund}")
        print()
    else:
        print()
        print("✗ Cancel aborted.")
        print()

```

```

def railway_services(user_name):
    while True:
        width = 45
        line = "=" * width

        print("+ " + line + "+")
        print("| " + " RAILWAY SERVICES ".center(width) + "|")
        print("+ " + line + "+")
        print("| " + "[1] View All Trains ".center(width) + "|")
        print("| " + "[2] Search Train by Route".center(width) + "|")
        print("| " + "[3] Book Ticket ".center(width) + "|")
        print("| " + "[4] My Bookings ".center(width) + "|")
        print("| " + "[5] PNR Status ".center(width) + "|")
        print("| " + "[6] Cancel Ticket ".center(width) + "|")
        print("| " + "[7] Logout ".center(width) + "|")
        print("+ " + line + "+")

        print()

        choice = input("Choose The Option: ")
        if choice == "1":
            view_trains()
        elif choice == "2":
            search_trains()
        elif choice == "3":
            book_ticket(user_name)
        elif choice == "4":
            view_bookings(user_name)
        elif choice == "5":
            pnr_status()
        elif choice == "6":
            cancel_ticket(user_name)
        elif choice == "7":
            print("Logging out...")
            break
        else:
            print("X Invalid option.")

# ===== Main Program =====

while True:
    try:
        width = 45
        line = "=" * width

        print("+ " + line + "+")
        print("| " + " WELCOME TO RAILWAY RESERVATION PORTAL ".center(width) + "|")
        print("+ " + line + "+")

        print("| " + "[1] Sign In".center(width) + "|")
        print("| " + "[2] Sign Up".center(width) + "|")
        print("| " + "[Q] Quit ".center(width) + "|")

        print("+ " + line + "+")

        print()

        option = input("Choose option: ").lower()
        if option == "1":
            print()
            SignIn()
        elif option == "2":
            print()
            SignUp()
        elif option == "q":
            print("You have exited the portal. Goodbye!")
            break
        else:
            print("X Invalid Option. Try again.")

    except ValueError:
        print("X Invalid input. Try again.")
        continue

```

7) Output:-

1.)Home Page

```
>>>
=====
Enter Database Password: 201207
☒ Database setup complete.
=====
| WELCOME TO RAILWAY RESERVATION PORTAL |
=====
|                                     |
|           [1] Sign In               |
|           [2] Sign Up               |
|           [Q] Quit                  |
|                                     |
=====
Choose option: |
```

2.)Sign Up Page

```
+=====+
| SIGN UP PAGE |
+=====+

Type 'quit' as the username to leave the sign-up page.

Username: Rohit0007
First Name: Rohit
Last Name: Kumar
Password: Rohit1234
Email: Rohit21@gmail.com
Phone Number: 9876543210

Captcha: 09KJA7
Enter the captcha as shown above: 09KJA7
☒ Captcha verified!

☒ Account created successfully!
+=====+
| WELCOME TO RAILWAY RESERVATION PORTAL |
+=====+
|                                     |
|           [1] Sign In               |
|           [2] Sign Up               |
|           [Q] Quit                  |
|                                     |
+=====+
Choose option: |
```

3.)Sign In Page

```
+=====+
| SIGN IN PAGE |
+=====+

Type 'quit' as the username to leave the sign-in page.

Username: Rohit0007
Password: Rohit1234

Captcha: OK78EG
Enter the captcha as shown above: OK78EG
☒ Captcha verified!

+=====+
| ☒ Welcome back, Rohit Kumar! |
+=====+
```

4.)Main Menu Page

```
+=====+
| ☒ Welcome back, Rohit Kumar! |
+=====+

+=====+
| RAILWAY SERVICES |
+=====+
| [1] View All Trains |
| [2] Search Train by Route |
| [3] Book Ticket |
| [4] My Bookings |
| [5] PNR Status |
| [6] Cancel Ticket |
| [7] Logout |
+=====+

Choose The Option:
```

5.)View All Trains Page

Choose The Option: 1			
Train No	Train Name	Route	Seats
1001	Rajdhani Express	Delhi → Mumbai	100
1002	Shatabdi Express	Delhi → Chandigarh	75
1003	Duronto Express	Mumbai → Kolkata	50

6.)Search Trains Page

```
Choose The Option: 2
Enter Source: Delhi
Enter Destination: Chandigarh

Matching Trains:
1002 | Shatabdi Express | Seats: 75
```

7.)Booking Page

```
+-----+
|                                     |
|               RAILWAY SERVICES     |
|                                     |
| [1] View All Trains                |
| [2] Search Train by Route          |
| [3] Book Ticket                    |
| [4] My Bookings                    |
| [5] PNR Status                     |
| [6] Cancel Ticket                  |
| [7] Logout                          |
|                                     |
+-----+

Choose The Option: 3
Enter Train No: 1002
Seats to book: 3

💻 Payment Gateway: Amount to pay ₹1500
Proceed with payment? (yes/no): yes
Processing payment...
☒ Payment successful!

💻 Ticket booked. Your PNR: STFUYIP8QP
```

8.)My Bookings Page

Choose The Option: 4						
Booking ID	PNR	Train	From	To	Seats	Time
1	STFUYIP8QP	Shatabdi Express	Delhi	Chandigarh	3	2025-11-22 18:56:50

9.)PNR Status Page

Choose The Option: 5

Enter your PNR: STFUYIP8QP

PNR Status:

PNR: STFUYIP8QP, Train: Shatabdi Express, From: Delhi, To: Chandigarh, Seats: 3, Time: 2025-11-22 18:56:50

10.)Cancellation Page

```
+=====+
|                                     |
|                               RAILWAY SERVICES                               |
|                                     |
| [1] View All Trains               |
| [2] Search Train by Route         |
| [3] Book Ticket                   |
| [4] My Bookings                   |
| [5] PNR Status                    |
| [6] Cancel Ticket                 |
| [7] Logout                        |
|                                     |
+=====+
```

Choose The Option: 6

Enter PNR to cancel: STFUYIP8QP

Confirm cancel? Refund ₹1500 (yes/no): yes

☒ Cancelled Sucessfully. Refund ₹1500

11.)Logout

```
+=====+
|                                     |
|                               RAILWAY SERVICES                               |
|                                     |
| [1] View All Trains               |
| [2] Search Train by Route         |
| [3] Book Ticket                   |
| [4] My Bookings                   |
| [5] PNR Status                    |
| [6] Cancel Ticket                 |
| [7] Logout                        |
|                                     |
+=====+
```

Choose The Option: 7

Logging out...

• Errors

1.) Inccorrent Phone Number

```
+=====+
|                SIGN UP PAGE                |
+=====+

Type 'quit' as the username to leave the sign-up page.

Username: Rohit0007
First Name: Rohit
Last Name: Kumar
Password: Rohit1234
Email: Rohit21@gmail.com
Phone Number: 987654321

❌ Invalid Phone Number, Try Again!
```

2.) Incorrect Username or Password

```
+=====+
|                SIGN IN PAGE                |
+=====+

Type 'quit' as the username to leave the sign-in page.

Username: Rohit0008
Password: Rohit1234

Captcha: G5TTQ1
Enter the captcha as shown above: G5TTQ1
☒ Captcha verified!
❌ Invalid username or password.
```

3.) Incorrect Captcha

```
Captcha: Q9IKY5
Enter the captcha as shown above: Q9IKY6
❌ Incorrect captcha.
❌ Incorrect captcha.
```

3.) Incorrect Train No, Train Location and PNR No.

```
Choose The Option: 2
Enter Source: Durg
Enter Destination: Delhi
❌ No trains found.
```

```
Choose The Option: 5
Enter your PNR: R0124RUHD
❌ Invalid PNR.
```

```
Choose The Option: 3
Enter Train No: 19487
Seats to book: 3
❌ Train not found or insufficient seats.
```

8) Limitations:-

Although the Railway Reservation System developed using Python IDLE and MySQL is functional and demonstrates core concepts of database connectivity and ticket management, it still has certain limitations.

User data is stored without encryption, making it unsuitable for handling sensitive information securely. Error-handling is basic, and the system does not prevent weak passwords or duplicate personal details.

Additionally, the project does not support advanced real-world features such as dynamic fare calculation, seat-wise allocation, real-time train status, or multi-user concurrency, which limits its use to educational purposes only. Since all processes operate sequentially, it cannot handle simultaneous users like real reservation systems.

Despite these limitations, the project effectively serves as a learning tool for understanding Python programming, MySQL operations, and backend logic.

9) Reference:-

S. No.	Type	Title / Source	Author / Organization	Website / Publication
1	Book	Computer Science with Python (Class 12)	Sumita Arora	Dhanpat Rai Publications
3	Website	Official Python Documentation	Python Software Foundation	https://docs.python.org
4	Website	MySQL Official Documentation	Oracle Corporation	https://dev.mysql.com/doc
5	Website	W3Schools Python Tutorial	W3Schools	https://www.w3schools.com/python
6	Website	W3Schools SQL Tutorial	W3Schools	https://www.w3schools.com/sql
8	Software	Python IDLE	Python Software Foundation	Installed locally
9	Software	MySQL & MySQL Workbench	Oracle Corporation	Installed locally

